

LogStack: A Smart Logging Stack Approach for IoT Devices based NDN (IoT-NDN)

Mohamed Ahmed M. Hail

Institute of Telematics

University of Lübeck

Ratzeburger Allee 160

Lübeck, Germany

hail@itm.uni-luebeck.de

Leon Dietrich

Institute of Telematics

University of Lübeck

Ratzeburger Allee 160

Lübeck, Germany

leon.dietrich@student.uni-luebeck.de

Stefan Fischer

Institute of Telematics

University of Lübeck

Ratzeburger Allee 160

Lübeck, Germany

fischer@itm.uni-luebeck.de

Abstract—On the one hand, the Internet of Things (IoT) is becoming more and more important in many areas of daily life and increasingly also in safety-critical applications; on the other hand, the resulting higher demand for the reliability of such applications can hardly be provided by the rather poorly resourced IoT devices themselves. In addition, alternative Internet architectures such as Named Data Networking (NDN) are increasingly being considered, which, on the one hand, greatly simplify the programming and use of Internet applications, but on the other hand, do not come close to the comprehensive tool infrastructure of the traditional Internet. To solve both problems, this paper proposes the realization of a Smart Logging Stack (SLS) that records the activities of IoT devices and makes them available for evaluation by appropriate applications. In this way, trust in the activity of an IoT device can be significantly increased. The SLS also fits into the IoT-NDN architecture, a variant of NDN for the IoT, adding an important component to its toolbox. The concept and implementation of SLS, as well as an overview of IoT-NDN, are described in this paper. Finally, some evaluation results are presented.

Index Terms—Internet of Things (IoT), Named Data Networking (NDN), Smart Logging Stack

I. INTRODUCTION

The number of IoT devices in use around the world is now growing rapidly every day. In industries such as smart cities, smart homes, smart mobility, etc., there are already very many functioning applications today. IoT devices monitor and observe objects, events or critical situations and usually deliver corresponding data, mostly on a large scale, to evaluation applications via the Internet. IoT devices are small and correspondingly severely resource-constrained; in particular, they have no permanent power supply, as well as low computing and storage capacities.

On the other hand, so-called Named Data Networking (NDN) is increasingly being investigated as a promising architecture for the future Internet. NDN uses names instead of IP addresses to request and transmit data. The IoT-NDN architecture has been designed, developed, and modified for IoT devices based on this NDN communication paradigm especially considering the limitations of IoT devices.

A major challenge in IoT applications remains the reliability of individual devices and thus the confidence in the validity of statements about the state of specific objects or the shape

of an important event. In applications related to environmental monitoring, this may often not be critical, but in home patient monitoring, for example, one wants to be sure to always have correct information available.

A common approach to increase confidence is to record (log) events in the devices and analyze the recorded data on a regular and timely basis. While logging solutions exist for traditional IP address-based architecture, to our knowledge this is lacking for the IoT-NDN-based Internet. We have therefore developed a Smart Logging Stack (SLS) that handles logging of states and events such as the state of built-in memory and power, as well as fatal errors that may occur on the device, on IoT-NDN devices.

As a major addition to IoT-NDN, an additional SLS table was added to store log messages, as well as several additional parameters. The SLS approach was programmed as a C/C++ library for the IoT-NDN architecture and evaluated and tested on the ESP32 development board.

The rest of this paper is organized as follows: after an overview of related work in Section II, the IoT-NDN architecture is presented in Section III. Section IV introduces the concept of the Smart Logging Stack. Section V discusses the implementation and evaluation, and Section VI summarizes the results of the work.

II. RELATED WORK

The evaluation of applications increases the performance and the availability of network devices. IoT devices are limited and resource-constrained. That makes logging mechanisms to optimize and evaluate network devices and applications very complex.

The next research articles [1], [2], [3], [4] and [5] are discussing the logging in IoT network and its devices.

The authors in [1] have discussed a secure management approach for logs in the Internet of Things (IoT). The advantages of managing logs on IoT devices have been investigated. An overview in [1] is presented to show how logs on IoT devices can be efficiently and securely stored.

The research work in [2] presents an approach called LogSafe, which efficiently and securely stores logs, including personal data from IoT devices. The LogSafe satisfies

the security properties such as Confidentiality, Integrity, and Availability (CIA).

The authors in [3] have presented a ProvThings approach that generates data provenance from IoT Apps and devices to provide a holistic explanation of system activities. They demonstrate the proposed approach in Samsung SmartThings and evaluate its efficiency and performance.

[4] presents a DisLog approach which is used for IoT forensics. The authors present this approach to aggregate, compress, and encrypt the logs on IoT devices. The approach also aims to secure the IoT log files.

An additional research work [5] discusses a novel logging approach for indoor localization in IoT environments, such as Smart Home. This approach uses machine learning and pattern recognition-based mechanisms, including a random forest-based classification model.

The next research works [6], [7] and [8] discuss general log mechanisms using the NDN protocol. The research work called Mnemosyne describes the design of a framework as a distributed logger based on Named Data Networking (NDN) [6]. The Mnemosyne project logs events and should assure immutability of the logged events using all event records in a DAG-mesh.

The research work mentioned in [7] considers the measurement of security and content dissemination and analysis of forwarding behavior-based NDN protocol. To reach such analysis, the authors present two mechanisms which are big data analysis and logs/traces generation. This solution generates big data logs that are essential for such analysis. The main goal of this paper is to evaluate the performance of the Named Data Networking Forwarding Daemon (NFD), which is an extension of the NDN protocol.

LogDoS [8] is a novel approach that uses Bloom-filter-based logging mechanisms to filter attack traffic efficiently. The proposed approach addresses the problem of data flooding attacks in Information-Centric Networks (ICN).

To the best of our knowledge, our research work is the first research work which is presenting a solution for IoT devices based NDN and records the activities and handles logging of states and events of IoT devices.

III. OVERVIEW OF IoT-NDN ARCHITECTURE

This section introduces the IoT-NDN architecture. All details and information about IoT-NDN architecture have already been published in [9]. The research articles [10], [11], [12] and [13] have already discussed the implementation of IoT-NDN. However, in this section, we will present the key complements of the IoT-NDN and show them shortly. Figure 1 summarizes all components of IoT-NDN system.

The IoT device layer is illustrated at the bottom of the IoT-NDN architecture, which includes all heterogeneous IoT devices. The top of the IoT-NDN architecture contains the IoT applications which are requesting the data. IoT-NDN operates as a networking layer, delivering the requested data from the underlying layer to the overlying layer.

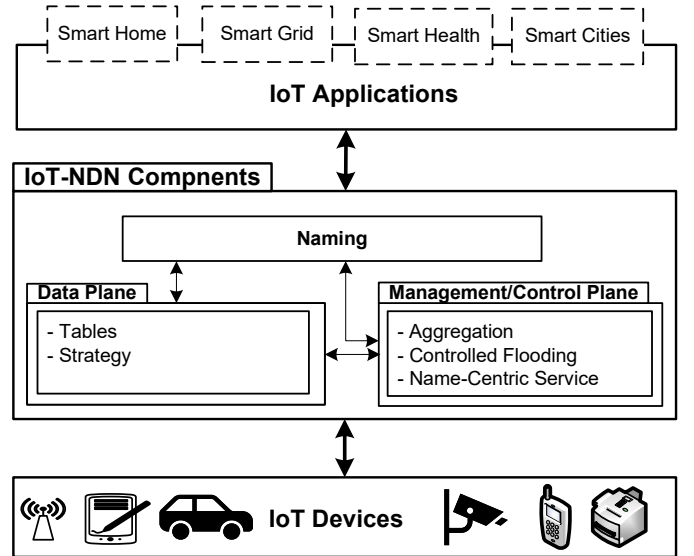


Fig. 1. IoT-NDN System architecture and its components

The IoT-NDN has three main components: Naming, Data Plane, and the Management and Control Plane. The naming component contains a new naming scheme and structure that is suitable for constrained wireless network devices. The management and control plane contains once again subcomponents such as Unicast Faces, Forwarding Services, Intra Node Protocol (INP), Controlled Flooding, Configuration, and Alias Service. The data plan component contains the strategy, which includes the caching and the forwarding strategies of IoT-NDN data. IoT-NDN components handle IoT-NDN packets and operating naming, caching, strategies, etc., in the IoT-NDN system. All components illustrated in Figure 1 are designed, developed, and added to match the IoT requirements.

All devices which use IoT-NDN architecture also have three tables; CS, PIT, and FIB. All IoT-NDN packets (Interest and Data packets) are received and sent through faces. In IoT-NDN, the faces are used concerning the network or local applications and simplify the processing and forwarding of messages in the IoT-NDN system. Face implementation for the IoT-NDN is also explained in details in [9].

More details about the IoT-NDN components and their functionalities can be found in [9] and [14].

IV. CONCEPT AND ARCHITECTURE

After introducing the IoT-NDN architecture in Section III, in this section, we will describe the concept and architecture of our approach.

IoT-NDN devices are often constrained by memory, energy, and computation power. IoT-NDN devices use most microcontrollers which are built and based on integrated circuits with a small processor and onboard memory with pins for input and output. In this paper, we will execute our experiments using the ESP32 Developer Board. More details about the ESP32 board is in [15].

The Data Plane of the IoT-NDN architecture has been extended with the Smart Logging Stack (SLS) table. The SLS is an additional table that caches entries of logged messages, e.g., logging of states and events such as the state of built-in memory and power, as well as fatal errors that may occur on the device, on IoT-NDN devices. Figure 2 shows the extended data plan of IoT-NDN architecture, including the SLS table illustrated as a gray complement.

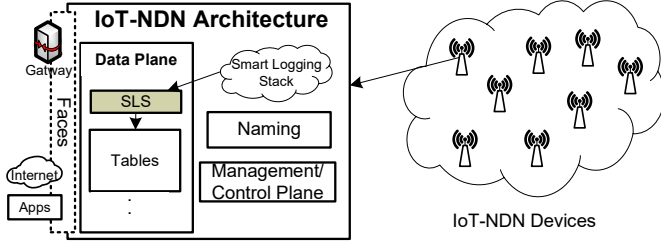


Fig. 2. The extended IoT-NDN Architecture with the SLS Approach

The log message contains two different parameters, namely log level and Scope. The log level is defined to prioritize the log messages. Some produced log messages can be used for collecting information about devices in the network, such as memory and energy status. But there are other log messages which are very important to be immediately sent to the gateway or access points, such as fatal errors or defect sensors on the device. We have categorized the priority of log messages as follows:

- **DEBUG = 0:** It is used for information that is needed to diagnose problems or fix bugs. Additionally, It is used when applications run in the test phase to optimize the system.
- **INFO = 10:** It is used to deliver general information about the application, device, or sensors running on the IoT device.
- **WARNING = 20:** It is used for situations or problem happened in the application or device that are unexpected but the code can continue working.
- **ERROR = 30:** It should be used when the application/system runs into one or more functionalities issues.
- **FATAL = 40:** it should be used when the application/system encounters an event or state in which one of the critical subsystems is no longer working.

One more feature of the SLS approach is the log message scope. The scope parameter defines the hop number of the log message, e.g., how many hops should the log messages (e.g., fatal errors, etc.) be forwarded in the network. There are different scopes values which are listed as follows.

- **Scope 0:** saving the log message locally. The log message will be cached in the SLS table without forwarding.
- **Scope 1:** forwarding the log message to the applications on the same device.
- **Scope 2:** forwarding the log message to all applications on the same device and to the neighborhood devices (one hope).

- **Scope 3:** forwarding the Log message globally to all devices in the network (unlimited hopes).

Each log message has two additional parameters: the age and send status parameters. The age parameter works as a counter of added log messages to the SLS table. The age counter will be incremented every time by adding a new log messages (see more information about it in section V).

The send status parameter shows if the log message has already been sent or not. If a log message has been created, the two parameters will also be initialized as default values. Afterward, they will be added to the SLS table, and the age and send status parameters will also be set depending on the configuration of application.

A. Name Structure

Considering the naming concept in IoT-NDN, for instance, an Interest name to request a log message or information about a device or sensor located on the device could look like in Figure 3, where "/" indicates the components and does not belong to the names. The name here contains different components as shown in Figure 3. Using the proposed name structure, including the command component, the SLS approach can easily request the wished log messages (e.g., fatal errors, etc.) and analyze its answer as a Data packet. This concept is beneficial for the IoT system since the IoT-NDN architecture requests data using human-friendly names (clear names)



Fig. 3. The Naming Structure of the SLS Approach Including the Command Component

We developed different commands for our approach, which could be extended or modified depending on the application. Some of the suggested commands are illustrated in Table I.

The Table I shows commands which are integrated into the SLS approach. The command list can be extended with different usable and needed commands for applications using IoT-NDN architecture. For example, the command AddLog can be used to add a log message to the SLS Table. This mechanism considers the eviction policies to replace the old entry in SLS table in case it is full. You can find more information about the eviction policies in the subsection V-B.

The second example of log commands is the delete command "DeleteLogsX", which is used to delete log message entries from the SLS table. X defines the number of the log message entries which should be deleted from the table, where

$$1 \leq X \leq n$$

, and n is the max size of the SLS table. Another command "SendDeleteLogMsgsX" is used to send the log message and then delete the entry from the SLS table. That could be helpful for some IoT-NDN devices and their application in which the

log messages are not more useful to be cached in the SLS table.

TABLE I
LIST OF FEW LOG COMMANDS WHICH ARE INTEGRATED INTO THE SLS APPROACH

Commands	Description
AddLogMsg	to add a standard log message, e.g. a parameter is not existing to the SLS Table
DeleteLogMsgsX	to delete the last x log messages from the SLS Table
SendLogsMsgsX	to send the last x log messages from the SLS Table
SendDeleteLogMsgsX	to send and then delete the last x log messages from the SLS Table

V. IMPLEMENTATION

The SLS approach is written as a library in modern C++. Since we are also targeting micro controllers (especially the ESP32 family, but also other microcontrollers), not all parts are completely high-level. This exception does not effect ports targeting more powerful systems like Raspberry Pi or desktop computers. This chapter will address some key points for developing very limited hardware, such as microcontrollers and their challenges. Table II summarizes the implementation components discussed in this section.

TABLE II
LIST OF THE IMPLEMENTATION COMPONENTS DESCRIBED IN THE IMPLEMENTATION SECTION

Components	Description
Memory Structure	discusses the limited memory of ESP32 and how it will be solved
Eviction Policies	presents the solution of used evictions policies if the SLS table is full
Log Message Handlers	discusses and explains how handlers can be used here, including the interface description
Log Message forwarding using IoT-NDN	presents the log message forwarding using the IoT-NDN and SLS approach
Debugging commands	describes how the buffer, including inset and delete log messages, is managed in the SLS approach
Experimental Hardware Setup	presents the hardware setup of ESP32 developing board used in this paper
Usage	describes how the SLS approach library could be used to record logs

A. Memory Structure

The primary issue with this class of IoT devices is the limited system memory. Even microcontrollers as big as most ESP32 suffer from heap fragmentation quite fast. As a result, we use fixed storage buffers for the actual log message entries and their metadata. The attached log message handlers are an exception to this rule, as they are stored in a linked list.

This is because we do not dynamically add new log message handlers (usually, this is done at boot time). Conversely, we expect an infinite amount of logged events while the device operates. Thus we can not rely on dynamic memory management for log message entries. The message's size and metadata

buffers default to 32 entries, but this can be adjusted using the `NDNLOG_LOG_BUFFER_SIZE` preprocessor constant if the use case requires it.

We are required to introduce this new table as we don't have control over the evictions within the CS table. On the other hand, we are not allocating more space than necessary on the content store as we don't need to cache our Interest packets beyond their delivery.

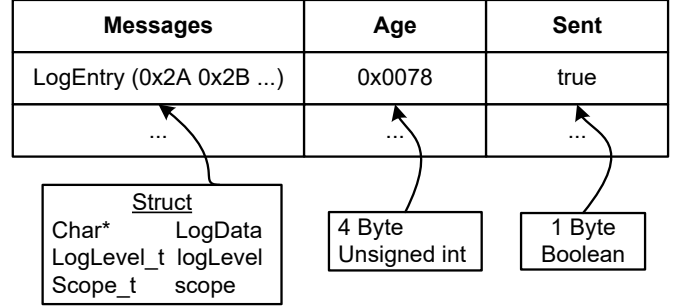


Fig. 4. The Structure of the SLS Table including the Log Message Entry Including Parameters

The log message buffer, as well as the metadata tables, are described in Figure 4. By keeping the scope and the log level within the message structure but separated, the message age and transmission index benefit from faster memory access times on our tested devices. This is due to them fitting in a single cache entry, causing a significant reduction in search times. Furthermore, the latter ones are considered to be an implementation detail, while the scope and log level of the entry need to be exposed to interfacing software, hence we won't suffer from overhead due to interface separation.

Meanwhile, the *message_ages* table stores the order in which entries have been created and the *broadcasted_messages* table keeps track of which of these entries were already successfully transmitted.

B. Eviction Policies

The library tries to find an empty spot in the buffer to store new messages first. As we are dealing with a fixed-size buffer, we also need to deal with the event that the buffer is completely filled with messages though. As a consequence, we will evict already transmitted messages, regardless of their log level first. This is considered to be safe as messages are only considered to be successfully transmitted if at least one of the utilized log handlers acknowledged one. Generally, a log handler will only signal a successful transmission if not only the data plane signaled a successful one but if it also received the acknowledged packet.

It is still possible that there won't be viable candidates (for example if no qualified logging receiver is reachable), thus we need to evict messages that have not been successfully transmitted yet. In this case, it will evict the oldest message from the (numerically) lowest log level it can find that is still lower than the log level of the new message to be stored. Assuming that all of this fails (i.g. the buffer is filled with not

yet successfully transmitted messages, each having a higher priority than the new message), the new message will not be stored in the buffer. An attempt to transmit it right away will be made neither the less.

C. Log Message Handlers

One would like to use different log handlers based on the situation. Different implementations for different platforms, like logging to networked servers, bare metal UART, VT-100 consoles, etc. are mandatory for such a system. As we cannot implement all possible use cases, we implemented a set of standard interfaces to attach to.

The most important interface is the *log_insert_handler* interface. It consists of a constant reference to the message to handle, as well a flag indicating, whether or not this is another attempt at transmitting the message. A method implementing this callback needs to return a boolean indicating a successful (or failed) transmission.

Furthermore, there exist interfaces used to query the stored messages, as well as an interface used for repopulating the buffer, after an unexpected power-off. The last one is only useful in conjunction with insert handlers that store the messages in non-volatile memory.

D. Log Message forwarding using IoT-NDN

The NDN default logging handler registers a packet handler with the *ndnph* library [16] at boot time. This handler is responsible to send Interest packets to the logging service. Unfortunately this cannot be done during runtime due to limitations of the used library. Next, the handler registers itself with the provided logger for the desired log level.

Given the event of a log message of sufficiently high level, the handler will create an Interest packet addressed at the configured logging service. It contains the message as a parameter. The logging service will then store the message for further processing and return the acknowledgment as a Data packet. Only this ACK package will then trigger the notation of a successful transmit. If it fails to send the Interest packet first hand it looks for other nodes nearby and transmits the package to them, hoping that those nodes are capable of delivery. This feature can be disabled by configuration though as it might impose a security risk.

This handler will also respond to packages sent to it without awaiting confirmation. It behaves like this as it needs to relay log messages from different nodes that cannot reach the logging service. As defined within table I, commands from the logging service are also handled this way.

In the event of a relay request, it will simply perform the same transmission action as it would have with one of its own messages. If the incoming package, however, reveals itself as a command from the logging service, it will simply call the corresponding method of the logger. This behavior needs to be separately enabled as well, due to the fact that we didn't have enough time yet to evaluate potential security implications.

E. Debugging commands

As we are testing the SLS approach, we need to manage the buffer and insert log messages effectively. In order to do so, we implemented a small command parser¹. These commands, however, are different from the ones described in table I, as they are invoked locally and feature pinging of other nodes, inspecting the log buffer and its metadata and logging messages.

Internally, these logger management commands are implemented as simple lambdas that attach to the interfaces mentioned above.

F. Usage

In order to use the logging library, one has to create an instance of *ndnlog::Logger*, either by calling the default constructor or the constructor to rebuild the buffer. As the logger is pretty useless without attached handlers, it would be wise to attach some right after construction. Naturally, there is a selection of existing handlers that we can automatically attach to a logger object. One that is capable of logging to UART and *ndnlog::register_default_ndn_callbacks* would be a suitable default. These methods, however, expect references to the interfaces being used; in this case, a reference to a UART stream or NDN Face.

As we are now set up, we can call *log* or *logf* (with *printf* like syntax) on our logger and watch it perform. The before-mentioned default NDN loggers will then handle the transmission as described in Figure 5.

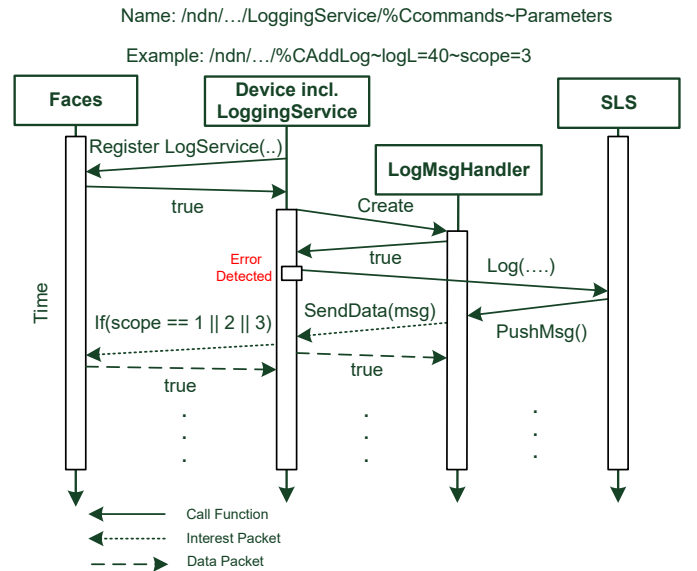


Fig. 5. A sequence diagram of log detecting using handler, SLS and Logging Service

¹In the future, we will use <https://github.com/patrislav1/linenoise-baremetal>, but for now, a simple buffer was sufficient and faster to integrate.

G. Experimental Hardware Setup

The logging library is used to aid the prototyping of IoT devices within our labs. Nevertheless, it is also required to test the library. In order to do so, we set up a couple of ESP32, as seen in Figure 6, used one to control the embedded neo pixels of the respective one. Besides that, our test environment only features LEDs connected to port 3, indicating an established connection as well as transmissions.

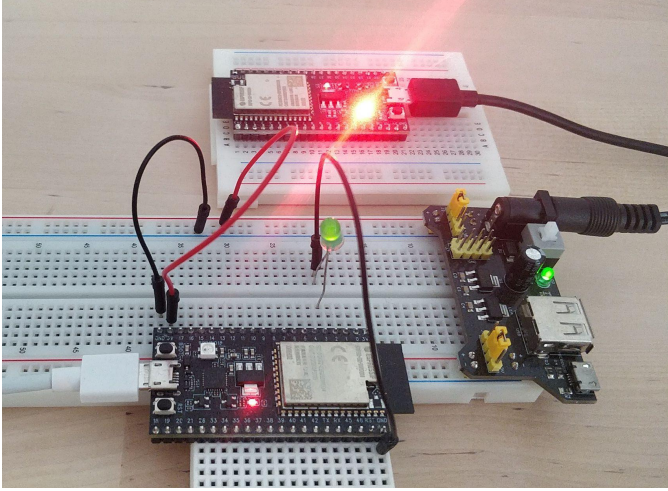


Fig. 6. The Experimental hardware setup including the ESP32

VI. CONCLUSION

In this paper, we discussed Named Data Networking (NDN) architecture which used names instead IP addresses for communication on the Internet. We introduced the IoT-NDN, a modified and extended architecture of NDN developed and designed for IoT devices considering the limitation.

We presented a proposed concept and architecture of the Smart Logging Stack (SLS) and integrated it into the IoT-NDN architecture. Using the naming structure of IoT-NDN, we defined a name structure for the SLS, which also integrates commands to request log messages from the devices. The SLS can record/log activities of IoT devices based NDN communication paradigm. The logged activities are available and can be used for evaluation or optimization by different applications/systems. We have shown that the trust activities on IoT-NDN devices can be significantly increased using the SLS approach.

The SLS has been implemented in C/C++ and tested on the ESP32 developing board. We include different parameters in the SLS implementation, such as log level (serves for prioritization) and Scope (serves for hops in the IoT-NDN network).

Finally, we tested our implementation using the ESP32 and implemented a small application to switch the LEDs of devices. After registering the LogService by the IoT-NDN faces, we used the debugging commands to request log messages from the device. Eviction policies have also been added to the

SLS to replace the log messages depending on the log level and lifetime if the memory is full.

In future work, we want to test the SLS on several platforms like raspberry pi, etc. We also will use the LogStack approach for security evaluation on IoT-NDN devices. In addition, we will do more experiments to optimize the approach.

REFERENCES

- [1] U. Gupta, "Secure management of logs in internet of things," *CoRR*, vol. abs/1507.05085, 2015. [Online]. Available: <http://arxiv.org/abs/1507.05085>
- [2] H. Nguyen, R. Ivanov, L. T. Phan, O. Sokolsky, J. Weimer, and I. Lee, "Logsafe: Secure and scalable data logger for iot devices," in *2018 IEEE/ACM Third International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2018, pp. 141–152.
- [3] Q. Wang, W. Hassan, A. Bates, and C. Gunter, "Fear and logging in the internet of things," 01 2018.
- [4] H. N. Noura, O. Salman, A. Chehab, and R. Couturier, "Distlog: A distributed logging scheme for iot forensics," *Ad Hoc Networks*, vol. 98, p. 102061, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1570870519306997>
- [5] N. Thakur and C. Y. Han, "Pervasive activity logging for indoor localization in smart homes," *2021 4th International Conference on Data Science and Information Technology*, 2021.
- [6] S. Liu, P. Moll, and L. Zhang, "Mnemosyne: an immutable distributed logging framework over named data networking," in *ICN '21: 8th ACM Conference on Information-Centric Networking, Paris, France, September 22 - 24, 2021*, G. Carofiglio, D. Oran, J. Ott, and L. Wang, Eds. ACM, 2021, pp. 130–132. [Online]. Available: <https://doi.org/10.1145/3460417.3483375>
- [7] J. Dongo, C. Mahmoudi, and F. Mourlin, "Ndn log analysis using big data techniques: Nfd performance assessment," 03 2018, pp. 169–175.
- [8] B. Al-Duwairi, Öznur Özkasap, A. Uysal, C. Kocaoğullar, and K. Yildirim, "Logdos: A novel logging-based ddos prevention mechanism in path identifier-based information centric networks," *Computers & Security*, vol. 99, p. 102071, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404820303448>
- [9] M. A. Hail, "Iot-ndn: An iot architecture via named data networking (ndn)," in *2019 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT)*, 2019, pp. 74–80.
- [10] M. A. Hail and S. Fischer, "Flexible API for IoT Services with Named Data Networking," in *2016 IEEE International Conference on Emerging Technologies and Innovative Business Practices for the Transformation of Societies (IEEE EmergiTech 2016)*, Mauritius, Mauritius, Aug 2016, pp. 1–6.
- [11] M. A. Hail, M. Amadeo, A. Molinaro, and S. Fischer, "Caching in named data networking for the wireless internet of things," in *Recent Advances in Internet of Things (RIoT), 2015 International Conference on*, April 2015, pp. 1–6.
- [12] M. A. Hail and S. Fischer, "Iot for aal: An architecture via information-centric networking," in *2015 IEEE Globecom Workshops (GC Wkshps)*, Dec 2015.
- [13] M. A. Hail, M. Amadeo, A. Molinaro, and S. Fischer, "On the performance of caching and forwarding in information centric networking for the iot," in *13th International Conference on Wired and Wireless Internet Communications*, April 2015.
- [14] I. Hail, Mohamed A.; Pösse and S. Fischer, "Integration of FIWARE and IoT based Named Data Networking (IoT-NDN)," in *In Proceedings of the 10th International Conference on Sensor Networks*, ISBN 978-989-758-551-7, ISSN 2184-4380, pages 184-190., 2022.
- [15] "Espressif systems: Esp32," <https://www.espressif.com/en/products/socs/esp32>.
- [16] "Named data networking packet headers," <https://github.com/yoursunny/NDNph>.